Bài 2.9 – Decision Tree (Cây quyết định)

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 2 buổi · Hình thức mở đầu: *Mini-game*

🎯 Sau bài này bạn sẽ

- Hiểu cây quyết định: một chuỗi câu hỏi "có/không", dễ hiểu như sơ đồ tư vấn.
- Tính tay **Gini impurity** và hiểu cây chọn câu hỏi thế nào.
- Huấn luyện, vẽ, đọc cây cho bài toán tín dụng và giá nhà.
- Kiểm soát overfitting bằng `max_depth`, `min_samples_leaf`.
- Hiểu vì sao cây **không cần chuẩn hóa** và đọc được **feature importance**.

Chia buổi:

- **Buổi 1:** Mục 1 → 4 (trực giác, Gini, cách chọn câu hỏi).
- **Buổi 2:** Mục 5 → 9 (cây phân loại, cây hồi quy, kiểm soát overfitting, bài tập).

---

1. Mini-game: Đoán món ăn trong 4 câu hỏi 🎮

Một người nghĩ trong đầu một món trong danh sách: **Phở, Bún chả, Cơm tấm, Bánh mì, Chè, Gỏi cuốn, Bánh xèo, Kem**. Bạn chỉ được hỏi câu **có/không**. Hỏi ít câu nhất để đoán ra.

Hai người chơi:

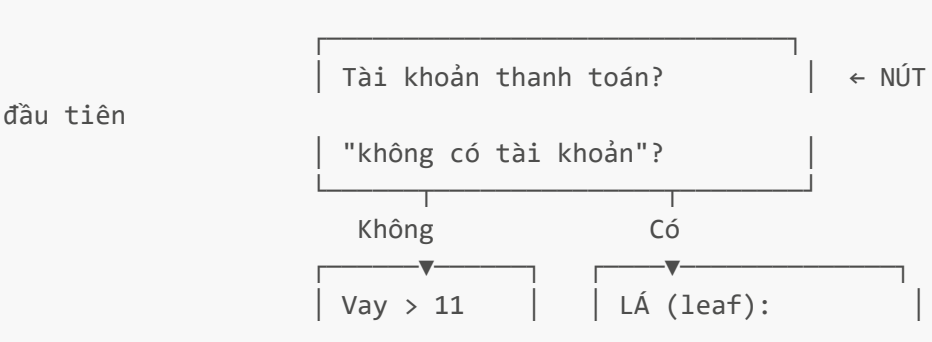
```
Người chơi A                              Người chơi B
"Có phải Phở không?"      Không           "Có phải món ngọt không?"   Không
"Có phải Bún chả không?" Không            "Có nước dùng không?"       Không
"Có phải Cơm tấm không?" Không            "Ăn kèm bánh tráng?"        Có
"Có phải Bánh mì không?" ...              "Có chiên giòn không?"      Có → Bánh xèo!
(có thể tới 7 câu)                        (luôn chỉ 3 câu)
```

Người chơi B thắng vì **mỗi câu hỏi chia đôi số khả năng**. Câu hỏi tốt là câu hỏi **tách nhóm sạch nhất**.

Decision Tree làm đúng như người chơi B: tự tìm câu hỏi tốt nhất ở mỗi bước, dựa trên dữ liệu.

---

2. Cấu tạo một cây



```
                    │ tháng?       │    │ dự đoán "trả tốt"  │
                    └──┬─────────┬─┘    │ (12% vỡ nợ)        │
                    Không      Có        └────────────────────┘
                     ...       ...      ← NÚT TRONG: câu hỏi tiếp theo
```

Thành phần	Ý nghĩa
Nút (node)	Một câu hỏi dạng "đặc trưng ≤ ngưỡng?"
Nhánh (branch)	Câu trả lời (có/không)
Lá (leaf)	Kết luận: lớp đa số (phân loại) hoặc giá trị trung bình (hồi quy)
Độ sâu (depth)	Số câu hỏi từ gốc tới lá xa nhất

Ví dụ đời thực: Sơ đồ "Bạn có nên mang ô hôm nay?" trên mạng xã hội. Hoặc kịch bản tổng đài: "Bấm 1 nếu..., bấm 2 nếu...".

---

3. Đo "độ lẫn lộn": Gini impurity

3.1. Khái niệm

- **Gini impurity (Độ vẩn đục Gini):** Đo mức độ **lẫn lộn** các lớp trong một nhóm. 0 = **thuần khiết** (chỉ có một lớp). Càng lớn càng lẫn lộn.
- **Ví dụ đời thực:** Một rổ toàn táo: Gini = 0. Một rổ nửa táo nửa cam: lẫn lộn nhất.

$$Gini = 1 - (p_1^2 + p_2^2 + ... + p_k^2)$$

Ký hiệu	Ý nghĩa
p_i	Tỉ lệ lớp i trong nhóm
k	Số lớp

Ý nghĩa xác suất: Gini = xác suất bốc ngẫu nhiên 2 lần (có hoàn lại) mà ra **hai lớp khác nhau**.

3.2. Tính thử

- 50% táo, 50% cam: 1 − (0,25 + 0,25) = **0,5** (lẫn lộn nhất với 2 lớp)
- 90% táo, 10% cam: 1 − (0,81 + 0,01) = **0,18**
- 100% táo: 1 − 1 = **0**

```python
def gini(ti_le):
    return 1 - sum(p ** 2 for p in ti_le)

print(round(gini([0.5, 0.5]), 3), round(gini([0.9, 0.1]), 3), gini([1.0]))
```

Kết quả mẫu:

```
0.5 0.18 0.0
```

---

4. Cây chọn câu hỏi thế nào?

Thuật toán (CART): Ở mỗi nút, thử **mọi đặc trưng** và **mọi ngưỡng** có thể. Chọn câu hỏi làm **Gini trung bình có trọng số** của hai nhóm con **giảm nhiều nhất**. Lặp lại với từng nhóm con, cho tới khi nhóm thuần khiết hoặc chạm giới hạn.

Tính tay với dữ liệu tín dụng

1.000 khách, 300 vỡ nợ (30%). Thử câu hỏi: **"Khách không có tài khoản thanh toán?"**

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_openml

credit = fetch_openml("credit-g", version=1, as_frame=True).frame
credit["vo_no"] = (credit["class"] == "bad").astype(int)
khong_tk = credit["checking_status"] == "no checking"
print(credit.groupby(khong_tk)["vo_no"].agg(["count", "sum", "mean"]).round(3))
```

Kết quả mẫu:

```
                 count  sum   mean
checking_status
False              606  254  0.419
True               394   46  0.117
```

Nhóm	Số khách	Tỉ lệ vỡ nợ p	Gini = 1 − p^2 − $(1-p)^2$
Trước khi chia	1.000	0,300	1 − 0,09 − 0,49 = **0,420**
"Không có TK" (Có)	394	0,117	≈ **0,206**
Có TK (Không)	606	0,419	≈ **0,487**

Gini sau khi chia (trung bình có trọng số theo số khách):

```
(394/1000) × 0,206 + (606/1000) × 0,487 = 0,081 + 0,295 = 0,376
```

Mức giảm Gini = 0,420 − 0,376 = **0,044**. Cây thử hàng trăm câu hỏi khác nhau và chọn câu hỏi có mức giảm lớn nhất. Với dữ liệu này, câu hỏi trên chính là câu hỏi ***ở gốc*** mà cây chọn (mục 5.2).

> Với hồi quy, thay Gini bằng **MSE**: chọn câu hỏi làm giá trị trong mỗi nhóm con **ít phân tán** nhất.

---

5. Cây phân loại: chấm điểm tín dụng

5.1. Chuẩn bị: dùng tất cả 20 cột

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import roc_auc_score

X = pd.get_dummies(credit.drop(columns=["class", "vo_no"]), drop_first=True,
dtype=int)  # one-hot mọi cột chữ
y = credit["vo_no"]
print(X.shape)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
```

Kết quả mẫu:

```
(1000, 48)
```

Không cần chuẩn hóa! Cây chỉ hỏi "duration ≤ 11,5?", câu hỏi này không đổi dù bạn đo theo tháng hay ngày. Thứ tự các giá trị mới quan trọng, không phải độ lớn.

5.2. Cây nông (max_depth = 3)

```
t3 = DecisionTreeClassifier(max_depth=3, random_state=42).fit(Xtr, ytr)
print(export_text(t3, feature_names=list(X.columns), show_weights=True))   # in
cây dạng chữ
```

Kết quả mẫu:

```
|--- checking_status_no checking <= 0.50
|   |--- duration <= 11.50
|   |   |--- purpose_education <= 0.50
|   |   |   |--- weights: [65.00, 11.00] class: 0
|   |   |--- purpose_education >  0.50
|   |   |   |--- weights: [1.00, 5.00] class: 1
|   |--- duration >  11.50
|   |   |--- savings_status_<100 <= 0.50
|   |   |   |--- weights: [89.00, 45.00] class: 0
|   |   |--- savings_status_<100 >  0.50
|   |   |   |--- weights: [127.00, 141.00] class: 1
|--- checking_status_no checking >  0.50
|   |--- other_payment_plans_none <= 0.50
```

```
|   |   |--- employment_4<=X<7 <= 0.50
|   |   |   |--- weights: [26.00, 16.00] class: 0
|   |   |--- employment_4<=X<7 >  0.50
|   |   |   |--- weights: [9.00, 0.00] class: 0
|   |--- other_payment_plans_none >  0.50
|   |   |--- employment_<1 <= 0.50
|   |   |   |--- weights: [219.00, 14.00] class: 0
|   |   |--- employment_<1 >  0.50
|   |   |   |--- weights: [24.00, 8.00] class: 0
```

Đọc cây như đọc quy định cho vay:

- `checking_status_no checking <= 0.50` nghĩa là "cột one-hot này bằng 0", tức khách **có** tài khoản thanh toán.
- `weights: [127, 141]` = trong lá này có 127 khách trả tốt, 141 khách vỡ nợ → dự đoán lớp 1 (vỡ nợ).
- **Nhánh rủi ro nhất:** có tài khoản thanh toán **VÀ** vay trên 11,5 tháng **VÀ** tiết kiệm dưới 100 DM → 141/268 ≈ **53% vỡ nợ**.
- **Nhánh an toàn nhất:** không có tài khoản thanh toán **VÀ** không có khoản trả góp khác → 14/233 ≈ **6% vỡ nợ**.

Đây là ưu điểm lớn nhất của cây: nhân viên tín dụng **đọc hiểu được ngay**, không cần biết toán.

> Muốn vẽ cây thành hình: `from sklearn.tree import plot_tree; plot_tree(t3, feature_names=X.columns, filled=True)`.

5.3. Feature importance (Độ quan trọng của đặc trưng)

```
quan_trong = pd.Series(t3.feature_importances_, index=X.columns)
print(quan_trong[quan_trong > 0].sort_values(ascending=False).round(3))
```

Kết quả mẫu:

```
checking_status_no checking    0.528
duration                       0.152
savings_status_<100            0.101
purpose_education              0.082
other_payment_plans_none       0.071
employment_4<=X<7              0.034
employment_<1                  0.032
dtype: float64
```

- **Định nghĩa:** Tổng mức giảm Gini mà mỗi đặc trưng đóng góp trên toàn cây (có trọng số theo số mẫu), chuẩn hóa cho tổng = 1.
- Câu hỏi về tài khoản thanh toán đóng góp **53%** "công sức" phân loại.

> ⚠️ Feature importance của cây đơn lẻ **không ổn định** và hay ưu ái các cột có nhiều giá trị khác nhau. Đừng tin tuyệt đối. Bài 2.10 có cách đáng tin hơn.

6. Overfitting: cây càng sâu càng "học vẹt"

```
for d in [1, 2, 3, 5, 8, None]:          # None: không giới hạn độ sâu
    t = DecisionTreeClassifier(max_depth=d, random_state=42).fit(Xtr, ytr)
    print(f"max_depth = {str(d):>4}: số lá = {t.get_n_leaves():>3} | acc train =
{t.score(Xtr, ytr):.3f}"
          f" | acc test = {t.score(Xte, yte):.3f} | AUC test = {roc_auc_score(yte,
t.predict_proba(Xte)[:, 1]):.3f}")
```

Kết quả mẫu:

```
max_depth =    1: số lá =   2 | acc train = 0.700 | acc test = 0.700 | AUC test =
0.683
max_depth =    2: số lá =   4 | acc train = 0.700 | acc test = 0.700 | AUC test =
0.711
max_depth =    3: số lá =   8 | acc train = 0.723 | acc test = 0.680 | AUC test =
0.689
max_depth =    5: số lá =  26 | acc train = 0.785 | acc test = 0.665 | AUC test =
0.682
max_depth =    8: số lá =  79 | acc train = 0.886 | acc test = 0.610 | AUC test =
0.509
max_depth = None: số lá = 151 | acc train = 1.000 | acc test = 0.620 | AUC test =
0.567
```

Bài 2.6 tái hiện hoàn hảo:

- Không giới hạn: cây mọc **151 lá** cho 800 khách, gần như **mỗi lá vài khách**. Train **100%**, test chỉ **62%** (thua cả mốc "đoán toàn trả tốt" 70%!).
- AUC ở độ sâu 8 là 0,509: **ngang tung đồng xu**.
- Cây tốt nhất ở độ sâu 2 (AUC 0,711). Vẫn thua Logistic Regression ở bài 2.5 (AUC 0,803).

Cây quyết định đơn lẻ là thuật toán dễ overfit bậc nhất. Cách kìm hãm:

Tham số	Ý nghĩa	Tác dụng
`max_depth`	Độ sâu tối đa	Giới hạn số câu hỏi liên tiếp
`min_samples_leaf`	Số mẫu tối thiểu ở mỗi lá	Cấm lá chỉ chứa vài mẫu (học thuộc từng khách)
`min_samples_split`	Số mẫu tối thiểu để được chia tiếp	Tương tự
`max_leaf_nodes`	Số lá tối đa	Giới hạn tổng độ phức tạp
`ccp_alpha`	Mức phạt độ phức tạp (cắt tỉa cây)	Giống regularization ở bài 2.6

```
tl = DecisionTreeClassifier(min_samples_leaf=20, random_state=42).fit(Xtr, ytr)
# mỗi lá ít nhất 20 khách
print(tl.get_n_leaves(), round(tl.score(Xtr, ytr), 3), round(tl.score(Xte, yte),
3),
      round(roc_auc_score(yte, tl.predict_proba(Xte)[:, 1]), 3))
```

Kết quả mẫu:

```
29 0.78 0.705 0.696
```

Chỉ còn 29 lá, train 78% (không còn học vẹt), test 70,5%.

---

7. Cây hồi quy: giá nhà

7.1. Cây chia "bậc thang"

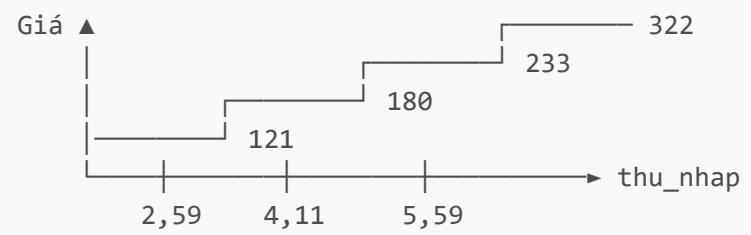
```
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error

df = nap_du_lieu_nha()          # ô chuẩn bị dữ liệu ở bài 2.1
Xtr, Xte, ytr, yte = train_test_split(df[["thu_nhap"]], df["gia_nha"],
test_size=0.2, random_state=42)
r = DecisionTreeRegressor(max_depth=2).fit(Xtr, ytr)
print(export_text(r, feature_names=["thu_nhap"]))
```

Kết quả mẫu:

```
|--- thu_nhap <= 4.11
|   |--- thu_nhap <= 2.59
|   |   |--- value: [120.82]
|   |--- thu_nhap >  2.59
|   |   |--- value: [180.24]
|--- thu_nhap >  4.11
|   |--- thu_nhap <= 5.59
|   |   |--- value: [233.04]
|   |--- thu_nhap >  5.59
|   |   |--- value: [322.49]
```

Mỗi lá dự đoán **giá trung bình** của các khu trong lá đó. Kết quả là một đường **bậc thang**, không phải đường thẳng:



Cây sâu hơn → bậc thang nhiều bậc hơn → bám dữ liệu sát hơn (và dễ overfit hơn).

7.2. Cây với 8 đặc trưng

```
cot = ["thu_nhap", "km_tp_lon", "phong_moi_ho", "ti_le_phong_ngu",
       "nguoi_moi_ho", "tuoi_nha", "kinh_do", "vi_do"]
Xtr, Xte, ytr, yte = train_test_split(df[cot], df["gia_nha"], test_size=0.2,
random_state=42)
for d in [3, 6, 10, 15, None]:
    r = DecisionTreeRegressor(max_depth=d, random_state=42).fit(Xtr, ytr)
    print(f"max_depth = {str(d):>4}: R² train = {r.score(Xtr, ytr):.3f} | R² test
= {r.score(Xte, yte):.3f}"
          f" | MAE test = {mean_absolute_error(yte, r.predict(Xte)):.1f}")
```

Kết quả mẫu:

```
max_depth =    3: R² train = 0.521 | R² test = 0.494 | MAE test = 51.2
max_depth =    6: R² train = 0.666 | R² test = 0.613 | MAE test = 43.2
max_depth =   10: R² train = 0.837 | R² test = 0.665 | MAE test = 37.9
max_depth =   15: R² train = 0.964 | R² test = 0.614 | MAE test = 39.3
max_depth = None: R² train = 1.000 | R² test = 0.589 | MAE test = 40.7
```

- Độ sâu 10: R^2 test **0,665**, MAE **37,9**: tốt hơn Linear Regression (0,537 / 48,7). Cây tự tìm ra quy luật phi tuyến và **tương tác** (ví dụ "thu nhập cao **và** gần thành phố"), mà không cần tạo đặc trưng đa thức.
- Không giới hạn: train R^2 = 1,0 (mỗi lá một khu!), test tụt xuống 0,589.

---

8. Ưu, nhược điểm

Ưu điểm	**Nhược điểm**
Dễ hiểu, dễ giải thích nhất trong mọi mô hình	**Rất dễ overfit**
Không cần chuẩn hóa	**Không ổn định**: đổi vài dòng dữ liệu, cây có thể thay đổi hoàn toàn
Tự nắm quan hệ phi tuyến và tương tác	Đường dự đoán bậc thang, không mượt

Ưu điểm	Nhược điểm
Chịu được outlier ở đặc trưng	Không dự đoán được ngoài khoảng đã thấy (giá cao nhất = lá cao nhất)
Nhanh	Một cây đơn lẻ thường thua các mô hình khác về độ chính xác

Tin tốt: Hai nhược điểm lớn nhất (overfit, không ổn định) được giải quyết bằng cách **trồng cả một khu rừng**. Đó là bài 2.10 (Random Forest) và 2.11 (Gradient Boosting), những mô hình **mạnh nhất** cho dữ liệu dạng bảng hiện nay.

---

9. Bài tập

1. **Tính tay** Gini của một nút có 40 khách: 10 vỡ nợ, 30 trả tốt. Nếu chia thành nhóm A (20 khách, 9 vỡ nợ) và nhóm B (20 khách, 1 vỡ nợ), Gini giảm bao nhiêu?
2. Theo cây ở mục 5.2, khách sau được dự đoán thế nào? Có tài khoản thanh toán, vay 24 tháng, tiết kiệm 500 DM.
3. Thử `DecisionTreeClassifier(max_depth=4, min_samples_leaf=30)` cho bài toán tín dụng. So sánh AUC test với độ sâu 2 và 3.
4. Thử `criterion="entropy"` thay vì Gini (mặc định). Kết quả có khác nhiều không?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. Nút gốc: p = 10/40 = 0,25 → Gini = 1 − 0,0625 − 0,5625 = **0,375**.
   - A: p = 9/20 = 0,45 → 1 − 0,2025 − 0,3025 = **0,495**
   - B: p = 1/20 = 0,05 → 1 − 0,0025 − 0,9025 = **0,095**
   - Sau chia: 0,5 × 0,495 + 0,5 × 0,095 = **0,295**. Giảm 0,375 − 0,295 = **0,08**.
2. Có tài khoản → nhánh trái. Vay 24 > 11,5 → nhánh `duration > 11.50`. Tiết kiệm 500 DM không thuộc nhóm "<100" → `savings_status_<100 <= 0.50` → lá `[89, 45]` → dự đoán **trả tốt** (rủi ro 45/134 ≈ 34%).
3. Code:

```
t = DecisionTreeClassifier(max_depth=4, min_samples_leaf=30,
random_state=42).fit(Xtr_c, ytr_c)
print(round(roc_auc_score(yte_c, t.predict_proba(Xte_c)[:, 1]), 3))
```

(Dùng lại X, y tín dụng ở mục 5.1; đặt tên biến khác vì mục 7 đã ghi đè `Xtr`, `Xte`.) Ghi lại kết quả. `min_samples_leaf` giúp cây sâu hơn mà không học vẹt, nhưng với 800 khách, cây đơn lẻ khó vượt AUC khoảng 0,7. 4. Entropy là một cách đo độ lẫn lộn khác (từ lý thuyết thông tin). Trong thực tế, Gini và entropy cho kết quả **rất giống nhau**. Gini tính nhanh hơn một chút nên là mặc định.

---

10. Flashcard

Mặt trước	Mặt sau
Decision Tree dự đoán thế nào?	Đi theo chuỗi câu hỏi có/không từ gốc tới lá; lá cho kết quả.
Gini impurity là gì?	$1 - \Sigma p_i^2$. Đo độ lẫn lộn các lớp. 0 = thuần khiết.
Gini lớn nhất với 2 lớp là?	0,5 (50/50).
Cây chọn câu hỏi thế nào?	Câu hỏi làm Gini trung bình có trọng số giảm nhiều nhất.
Cây hồi quy dự đoán gì ở lá?	Trung bình giá trị các mẫu trong lá.
Cây có cần chuẩn hóa?	Không.
Tham số chống overfit chính?	`max_depth`, `min_samples_leaf`.
Cây không giới hạn độ sâu thì sao?	Train 100%, test kém: học vẹt.
Nhược điểm lớn của cây đơn lẻ?	Dễ overfit, không ổn định.
Cách khắc phục?	Ghép nhiều cây: Random Forest, Gradient Boosting.

Bài tiếp theo: 2.10 – Random Forest.